Highlights

Gradient at the Edge of Chaos: Why Analytic-Gradient Reinforcement Learning Has No Seed-Robust Advantage on Chaotic Kolmogorov Flow

Cem Özkurt, Muhammed Emin Onay, Eyüp Altuğ Tunç, Ahmet Kutay Küçükler, Haktan Yusuf Güloğlu

- GSAC is the first off-policy analytic-gradient RL applied to Kolmogorov flow.
- At $Re = 45$, GPPO reproduces the reported sample-efficiency gain (63–84%).
- At $Re = 300$ all four algorithms diverge; the breaking episode is seed-dominated.
- An exact permutation test separates no pair of algorithms (all $p \geq 0.19$).
- The chaotic-regime break is sudden, precursor-free and replay-buffer gated.

Gradient at the Edge of Chaos: Why Analytic-Gradient Reinforcement Learning Has No Seed-Robust Advantage on Chaotic Kolmogorov Flow

Cem Özkurt[a,*], Muhammed Emin Onay[b], Eyüp Altuğ Tunç[b], Ahmet Kutay Küçükler[c] and Haktan Yusuf Güloğlu[c]

[a]Department of Data Science and Analytics, Faculty of Computer and Information Sciences, Sakarya University, Esentepe Campus, Serdivan, 54187, Sakarya, Türkiye

[b]Department of Computer Engineering, Faculty of Computer and Information Sciences, Sakarya University, Esentepe Campus, Serdivan, 54187, Sakarya, Türkiye

[c]Department of Computer Engineering, Sakarya University of Applied Sciences, Esentepe Campus, Serdivan, 54050, Sakarya, Türkiye

ARTICLE INFO

Keywords:
analytic policy gradients
differentiable simulation
reinforcement learning for flow control
chaotic dynamics
training stability
Kolmogorov flow

ABSTRACT

Does injecting an analytic (backpropagated-through-the-simulator) policy gradient help or hurt as a fluid-flow control task becomes chaotic, and does off-policy reinforcement learning (RL) tolerate the resulting gradient instability better than on-policy RL? The HydroGym paper reports that Gradient-Enhanced PPO (GPPO) cuts training iterations by at least 65 % relative to PPO, shown at a single Reynolds number and seed. We evaluate PPO, GPPO, SAC, and a gradient-enhanced SAC variant (GSAC), the first transfer of off-policy maximum-entropy analytic-gradient RL to fluid dynamics, on the real `hydrogym.jax` Kolmogorov-flow environment at a regular ($Re = 45$) and a chaotic ($Re = 300$) operating point over five seeds. At $Re = 45$, GPPO reaches PPO's best reward in substantially fewer episodes in 4 of 5 seeds, reproducing the reported mechanism. At $Re = 300$, all four algorithms eventually diverge, and the first-divergence episode is dominated by the seed, not the algorithm: an exact permutation test separates no pair of algorithms (all $p \geq 0.19$). The reproducible findings are mechanistic: the break is sudden and precursor-free; the replay buffer's size at the trigger determines whether a brief post-trigger survival window can appear; and in a rare mode the flow diverges within about two episodes regardless of algorithm. Analytic-gradient injection thus imposes no seed-robust ordering on chaotic-regime training stability.

1. Introduction

The active control of fluid flows — for drag reduction, mixing enhancement, or extreme-event suppression — is one of the areas where machine learning has changed computational fluid dynamics [1, 2] most directly, and reinforcement learning (RL) is now one of its most active application routes. Rabault et al. [3] first showed that deep RL can control periodic vortex shedding behind a cylinder, achieving ∼8 % drag reduction at $Re = 100$; the line has since extended into the turbulent regime, e.g. turbulent separation-bubble control [4] and real-time state-dependent turbulent-wake control [5]. HydroGym [6] consolidates these scattered codebases into a single solver-agnostic platform spanning 61+ environments from laminar to turbulent and — alongside classical solvers — ships a JAX-based, end-to-end *differentiable* pseudo-spectral solver that continues the line of differentiable-CFD libraries JAX-CFD [7] and JAX-Fluids [8, 9]. A methodologically related, PyTorch-based differentiable flow-control benchmark, FluidGym [10], belongs to the same ecosystem.

Differentiability lets RL move beyond model-free training by computing *analytic policy gradients*: the derivative of the reward with respect to the policy parameters, obtained by the chain rule *through the simulator itself* rather than by sampling. The HydroGym paper demonstrates this with GPPO, in which the analytic gradient is mixed with PPO's likelihood-ratio gradient — a construction closely related to Gradient-Informed PPO [11] — and on the Kolmogorov-flow environment this is reported to cut training iterations by at least 65 % and to yield controllers with lower action amplitude [6]. That result, however, is shown at a *single operating point* of the wide $Re = 40$–500 range the paper defines for Kolmogorov flow; the behaviour of the analytic gradient at the regular and chaotic ends of that range is not separately compared.

This gap overlaps a documented risk. Metz et al. [12] demonstrate a common "chaos-induced failure mode" of backpropagation through iterated maps, physics simulations, and learned optimizers, and tie it to the spectrum of the system Jacobian. On the fluid-dynamics side, Boffetta and Musacchio [13] show that the largest Lyapunov exponent of homogeneous-isotropic turbulence grows with Reynolds number, so the predictability horizon shrinks as Re rises. Together these two independent results suggest that in a chaotic regime the norm and direction of the analytic gradient can become hypersensitive to initial conditions, and that GPPO's reported advantage could weaken, or reverse, as Re increases.

*Corresponding author.
cemozkurt@sakarya.edu.tr (C. Özkurt)
ORCID(s): 0000-0002-1251-7715 (C. Özkurt); 0009-0000-0454-4986 (M.E. Onay); 0009-0007-5166-8077 (E.A. Tunç); 0009-0007-0202-016X (A.K. Küçükler); 0009-0007-9429-6154 (H.Y. Güloğlu)

Analytic-gradient injection has so far been applied mostly with *on-policy* methods or in rigid-body robotics tasks: SHAC [14] mitigates the exploding/vanishing gradient problem with short-horizon rollouts and a learned terminal value; AHAC [15] adapts the horizon to avoid stiff dynamics. On the off-policy side, SAPO [16] carries analytic-gradient RL into a maximum-entropy (SAC-like) framework and reports strong performance on deformable-body tasks — but this approach has never been applied to fluid dynamics. We add GSAC, a gradient-enhanced version of SAC [17], as a fourth method, placing the comparison at the intersection of the SHAC family (short-horizon analytic gradient) and the SAPO family (off-policy maximum-entropy analytic gradient), and asking whether the off-policy replay-buffer structure tolerates chaotic-regime gradient instability better than on-policy GPPO.

The paper makes the following contributions:

1. We extend HydroGym's GPPO-vs-PPO result on Kolmogorov flow [6] from a single Reynolds point to a regular-vs-chaotic regime comparison, and from $n = 1$ to $n = 5$ seeds (Section 5).
2. We introduce GSAC (SAC + short-horizon analytic gradient), never previously applied to HydroGym and the first transfer of the SAPO [16] family to fluid dynamics, giving a 2×2 design (on-/off-policy $\times$ with/without analytic gradient) that isolates each factor (Section 3).
3. *A seed-dominance result, replicated over five seeds with a formal test.* In the chaotic regime all four algorithms eventually diverge, and the episode at which they first diverge is dominated by the training seed, not the algorithm: each algorithm's first-NaN episode spans more than an order of magnitude, PPO is both the first to break at seed $= 0$ and the most robust of the four at seed $= 2$, and an exact sign-flip permutation test separates no pair of algorithms (all $p \geq 0.19$; Kruskal–Wallis $p = 0.57$, Friedman $p = 0.77$). Analytic-gradient injection does not delay the on-policy break (GPPO−PPO mean -2.4 episodes, $p = 0.81$).
4. *Two mechanistic results that do survive*, both algorithm-design-independent: (a) a replay-buffer dilution calculation for whether a brief post-trigger "survival window" can appear (non-zero probability only for large-buffer GSAC; one window seen in 20 runs, that case); and (b) an early-init failure mode — when the initial policy already sits near the solver's stability boundary the flow diverges within $\sim$2 episodes regardless of algorithm — documented directly with per-episode action-amplitude and analytic-gradient-norm logs (Section 6).
5. GSAC is characterized as highest ceiling, no reliable floor: 2–10$\times$ the stable horizon of any other method when training gets a foothold, but episode-2 failure when the flow diverges before learning begins.

We pre-registered the hypothesis that the norm/variance of the analytic gradient would be stable at low Re and unstable at high Re; against this instability, GSAC's off-policy structure would be more robust than GPPO's on-policy structure. At $n = 5$ this was not supported at $p < 0.05$: GSAC has the highest median breaking point and is most robust in 3 of 5 seeds, and GSAC−GPPO is positive in the mean (+39 episodes), but the sign-flip permutation p-value is 0.19 and the effect is dominated by seed-to-seed variance. Per the pre-registered decision rule (Section 5.3) the finding stands as "no seed-robust ordering"; GSAC's lean is reported as a candidate effect for a larger-n study.

The remainder of the paper is organized as follows. Section 2 reviews analytic-gradient RL and chaotic-system instability. Section 3 formulates the environment, the four algorithms, and the analytic-gradient mechanism with its computation graph and algorithm boxes. Section 4 gives the configuration, seed protocol, and reproducibility package. Section 5 reports the regular- and chaotic-regime results and the significance test; Section 6 dissects the break; Sections 7 and 8 discuss and conclude.

2. Related work

Table 1 positions this study against the closest prior work along three axes: the base RL family, whether an analytic (through-simulator) gradient is used, and whether the chaotic/turbulent regime is addressed.

2.1. RL for flow control and HydroGym

Since Rabault et al. [3], model-free RL has repeatedly succeeded on turbulent flow control: separation-bubble control [4] reports $\sim$9 % vs $\sim$6.8 % reduction over classical periodic forcing; an early study [20] establishes DRL's applicability to turbulent conditions; real-time state-dependent bluff-body wake control [5] achieves 2–4$\times$ the performance of model-based baselines in wind-tunnel experiments; deep RL has also learned energy-efficient collective swimming gaits by exploiting shed vortices [21]. A broad robotics survey [22] documents the wider maturity of RL for real-world control. HydroGym [6] unifies these under one solver-agnostic interface, shows zero-shot policy transfer across environments, and reports the $\geq$65 % sample-efficiency gain for GPPO over PPO on Kolmogorov flow that this paper examines. FluidGym [10] is a related PyTorch-based differentiable benchmark with PPO/SAC baselines. None of these works systematically compares analytic-gradient injection across a regular and a chaotic regime.

2.2. Analytic policy gradients through differentiable simulation

The single closest prior method to GPPO is GI-PPO [11]: it introduces an "α-policy" that adaptively weights the analytic gradient against the PPO gradient, down-weighting the analytic term when it is detected to be high-variance or biased. Our fixed 50/50 mix is the non-adaptive special case (α held constant). RPO [18] analyzes the on-policy analytic-gradient setting directly, showing that reparameterization policy gradients with sample reuse optimize a PPO-style

Table 1
Positioning against the closest prior work. AG = analytic (through-simulator) policy gradient; ME = maximum-entropy (SAC-style) objective; ✓ = present, dash = absent.

Work	Base	AG	ME	Chaotic	Domain
HydroGym / GPPO [6]	PPO	✓	–	one Re	Kolmogorov flow
GI-PPO [11]	PPO	✓(adapt.)	–	–	rigid body
RPO [18]	PPO	✓	–	–	control suites
SHAC [14]	AC	✓(short-H)	–	–	locomotion
AHAC [15]	AC	✓(adapt.-H)	–	–	contact-rich
SAPO [16]	SAC	✓	✓	–	rigid/deformable
Bucci et al. [19]	DDPG	–	–	✓	Kuramoto–Sivashinsky
Font et al. [4]	PPO	–	–	✓	separation bubble
This work	PPO, SAC	✓	✓	45 / 300	Kolmogorov flow

surrogate via backpropagation-through-time (BPTT) and targeting exactly the training instability this paper measures at $Re = 300$. The broader family originated in robotics/control. SHAC [14] decouples the analytic gradient from the full episode horizon using short-horizon rollouts and a smooth critic; our 6-step horizon is a direct adaptation. AHAC [15] replaces the fixed horizon with one adaptively truncated to avoid stiff dynamics. SAPO [16] carries the idea into a maximum-entropy, off-policy framework, testing whether entropy regularization smooths the analytic-gradient landscape; our GSAC is the first application of the SAPO idea to fluid dynamics.

2.3. Analytic-gradient instability in chaotic systems

The theoretical basis of our hypothesis is Metz et al. [12]: in chaotic dynamical systems, backpropagated gradients explode through a butterfly-effect mechanism rooted in the system Jacobian spectrum, with clipping, truncated BPTT, or a switch to zeroth-order methods proposed as remedies. Parmas et al. [23] show the same failure directly in model-based policy search: gradients through long chains of nonlinear computation become large and effectively random in direction, and reparameterization (analytic) gradients suffer from it while likelihood-ratio (model-free) gradients are comparatively robust — the same asymmetry this paper's on-policy result partially echoes (Section 5). The fluid-dynamics counterpart is Boffetta and Musacchio [13]: the largest Lyapunov exponent (estimated in practice via the classical time-series method of Wolf et al. [24]) of homogeneous-isotropic turbulence grows with Re and the predictability horizon shrinks inversely with it. Together these frame our 500-episode finding that GPPO and GSAC also eventually reach NaN, only later: instability is inevitable; what differs is only the delay mechanism. That gradient-free RL *can* control spatiotemporal chaos is not in question — Bucci et al. [19] do exactly this on the Kuramoto–Sivashinsky equation — the question here is whether *analytic-gradient* injection survives the chaotic regime.

2.4. Analytic vs. zeroth-order gradients; partial differentiability

Suh et al. [25] compare analytic and zeroth-order policy gradients in systems with stiffness and discontinuities, and via a bias–variance analysis propose an α-graded interpolating estimator, of which our 50/50 mix is roughly the $\alpha = 0.5$ special case. A 2026 follow-up [26] re-tests the same question with a lighter hybrid-switching scheme. ABPT [27] proposes an amended BPTT scheme combining 0-step and N-step returns for partially differentiable rewards, a problem class close to HydroGym's mixed control-reward / physics-state structure.

2.5. Base algorithms and distributed PDE control

The gradient-free anchors are PPO [28] and Soft Actor-Critic [17, 29]. PPO builds on TRPO [30] and Generalized Advantage Estimation (GAE) [31]; Soft Q-learning [32] is the origin of SAC's maximum-entropy framework; both algorithms' updates in this study use Adam [33]. On the distributed structure of PDE actuation, Peitz et al. [34] exploit translational equivariance to turn a high-dimensional control problem into a multi-agent problem with identical uncoupled agents; Kolmogorov flow's low-wavenumber sinusoidal actuators sit in the same distributed-control setting.

3. Methods

3.1. Approach

Model-free reinforcement learning treats the simulator as a black box and estimates the policy gradient from sampled returns alone. This is robust and makes almost no assumptions, but the estimate is high-variance and sample-hungry, and on flow-control problems it can take tens of thousands of environment interactions to converge. When the simulator is itself differentiable, a second route becomes available: differentiate the accumulated reward *through* the solver and obtain the policy gradient from the chain rule directly. This *analytic* (or first-order) gradient is far lower in variance and, on smooth control problems, converges in a fraction of the interactions. It is

the mechanism behind differentiable-physics methods such as SHAC [14], AHAC [15] and SAPO [16] in robotics, behind differentiable-CFD control with JAX-CFD [7] and JAX-Fluids [8], and behind the GPPO result on Kolmogorov flow that this paper follows up [6].

The derivative that makes this route fast also makes it fragile. Backpropagating through a long or stiff rollout produces gradients that explode or vanish, and the estimate is biased whenever the horizon is truncated or the dynamics are contact-rich or chaotic [12, 25]. The established mitigations are to use a *short* differentiation horizon, to forward-propagate only the deterministic mean action so the differentiated chain stays smooth, and to *mix* the analytic gradient with the ordinary model-free gradient rather than trust it alone [11, 14, 25]. Chaotic flow is exactly where this fragility is expected to matter most: neighbouring trajectories separate exponentially, so the through-solver gradient of a many-step rollout can be swamped by noise, and the predictability horizon shrinks as the Reynolds number rises [12, 13]. Whether the analytic gradient still helps in that regime — and whether an off-policy replay buffer absorbs the instability better than an on-policy update — is the question this study is built to answer.

The rest of this section constructs the four algorithms used to test it. We fix a single environment and control problem (Section 3.3) and cast it as a Markov decision process (Section 3.4); we state the two model-free estimators, PPO and SAC (Sections 3.5 and 3.6); we then define the analytic gradient and the fixed 50/50 mixing rule (Section 3.7); and finally we assemble the on-policy variant GPPO (Section 3.8) and the off-policy variant GSAC (Section 3.9). The factorial design that crosses these choices is described first.

3.2. Overall design

We compare four algorithms in a 2×2 factorial design that crosses the RL family (on-policy vs. off-policy) with analytic-gradient injection (absent vs. present), so that each factor and their interaction can be read off directly (Fig. 1). All four share one environment, one network architecture, one optimizer, and one training budget (Section 4); they are evaluated at a regular ($Re = 45$) and a chaotic ($Re = 300$) operating point of the same Kolmogorov-flow solver.

3.3. Environment and control problem

Experiments run on the real `hydrogym.jax.envs.kolmogorov.KolmogorovFlow` environment: a 64×64 grid (4096 degrees of freedom), pseudo-spectral IMEX-RK4CN time integration [6, 7], with four low-wavenumber sinusoidal forcing modes as actuators (Fig. 2). The 2D incompressible Navier–Stokes equations in vorticity form are integrated as $\omega_{t+1} = f_{\text{sim}}(\omega_t, a_t; Re)$, where the discrete operator f_{sim} is written in JAX [35] and is differentiable end-to-end.

Two library quirks required a subclass. (i) The environment step ignores the per-call time-step and action-duration arguments and re-reads its own `default_params`, so we override that property on a subclass (`FastKolmogorovFlow`) to reach proof-of-concept scale. (ii) The library default $\alpha_{\text{reward}} = 0$ makes the reward control-cost only; we set $\alpha_{\text{reward}} = 1$ (Fig. 2).

At this grid/dt combination (64×64, d$t = 0.001$) the solver was empirically observed to diverge numerically near $Re \approx 350$–380 — even under zero action from purely analytic initial conditions, and not resolved by reducing dt tenfold (so not a simple CFL issue). We therefore use $Re = 300$ for the chaotic regime — the highest value that stays below this limit while forming a meaningful contrast with $Re = 45$ — rather than the originally intended $Re = 450$.

3.4. Markov decision process

The control task is a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with $s_t \in \mathbb{R}^{64}$, $a_t \in \mathbb{R}^4$, deterministic transition $s_{t+1} = g\big(f_{\text{sim}}(\omega_t, a_t; Re)\big)$ (g the observation map), reward as in Fig. 2, and discount $\gamma = 0.99$. An episode is $T = 15$ environment steps from a fixed reset state s_0. The policy $\pi_\theta(a \mid s)$ and the value / Q networks are 2-hidden-layer MLPs (64 units, tanh). The objective is the discounted return

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t\, r(s_t, a_t)\right]. \tag{1}$$

All four algorithms optimize Eq. (1); they differ only in the gradient estimator they apply to it.

3.5. PPO: on-policy baseline

PPO is an on-policy policy-gradient method: it collects one rollout under the current policy, estimates each action's advantage along it, and takes a single trust-region-style step that is clipped to stay close to the policy that generated the data. The rollout is then discarded, so PPO consumes each unit of environment experience through exactly one gradient update.

The clipped surrogate is due to Schulman et al. as a first-order approximation to the TRPO trust region [28, 30], and the Generalized Advantage Estimator (GAE) used to form the advantage is from the same group [31]. All gradient steps in this study, for every algorithm, use Adam [33], as in the original PPO and HydroGym [6] implementations.

The policy is a diagonal Gaussian: the MLP produces a bounded mean $a_\mu(s) = 0.5 \tanh(\cdot)$ and a state-independent learnable $\log \sigma$; the executed action is $a \sim \mathcal{N}(a_\mu(s), \sigma^2)$ and is *not* re-clipped, so sampled $|a|$ can exceed 0.5 (observed up to ≈ 1.2 early in training). Advantages use GAE [31]:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{2}$$

with $\lambda = 0.95$. PPO maximizes the clipped surrogate [28], derived from the TRPO trust region [30]:

$$\begin{aligned} \mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t\Big[& \min\big(\rho_t(\theta)\hat{A}_t, \\ & \quad \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \\ \rho_t(\theta) = & \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \end{aligned} \tag{3}$$

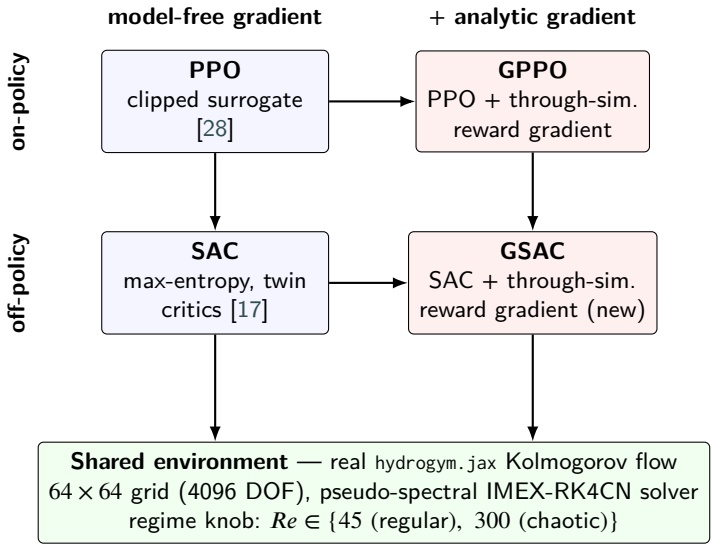


Figure 1: The 2×2 experimental design. Horizontal edges add the analytic (backpropagated-through-the-simulator) reward gradient; vertical edges switch the RL family. GSAC (bottom right) is introduced here. All four algorithms are run at both Reynolds numbers with identical budgets.

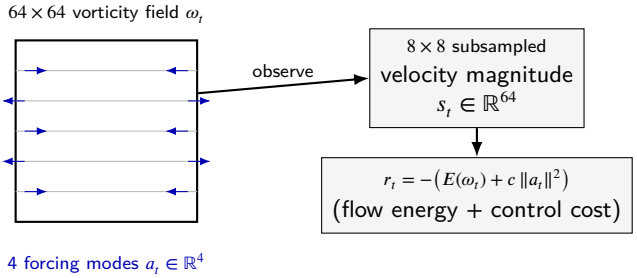


Figure 2: Kolmogorov-flow control problem. The policy observes an 8×8 subsampled velocity magnitude ($s_t \in \mathbb{R}^{64}$), commands four sinusoidal forcing-mode amplitudes ($a_t \in \mathbb{R}^4$), and is rewarded for reducing flow energy at bounded control cost. The library default $\alpha_{\text{reward}} = 0$ (control-cost only) is overridden to $\alpha_{\text{reward}} = 1$ so that flow energy enters the reward.

with $\epsilon = 0.2$. The total loss adds a value term and an entropy bonus,

$$\begin{aligned} \mathcal{L} &= -\mathcal{L}^{\text{PPO}} + 0.5\,\mathcal{L}^{V} - 0.001\,\mathcal{H}[\pi_\theta], \\ \mathcal{L}^{V} &= \mathbb{E}_t\big[(V_\phi(s_t) - \hat{R}_t)^2\big]. \end{aligned} \tag{4}$$

Our loop performs *one full-batch gradient step per episode* on Eq. (4) — no PPO epochs, no minibatching. This is a deliberately minimal PPO: it keeps the on- and off-policy loops comparable and the analytic-gradient mix simple, but "PPO" here is weaker than a fully tuned implementation. The comparison of interest is relative (PPO vs. GPPO, SAC vs. GSAC), and all four share this budget.

We use PPO as the on-policy baseline, rather than a different on-policy algorithm, because it is exactly the update the HydroGym paper's GPPO result is built on top of [6]: answering whether that result survives a chaotic regime requires holding the base update fixed and changing only the analytic-gradient mixing (Section 3.8), not also switching algorithms. Its clipped surrogate also gives a clean model-free-only control arm for the on-policy row of the 2×2 design (Fig. 1).

3.6. SAC: off-policy baseline

SAC is an off-policy, maximum-entropy actor-critic method: transitions are stored in a replay buffer and resampled many times, so one environment step contributes to many gradient updates instead of one. Its actor is a squashed Gaussian whose action is drawn by reparameterized sampling, so the entropy-regularized objective can be differentiated through the sampled action directly rather than estimated by a likelihood-ratio term.

SAC and its maximum-entropy objective are due to Haarnoja et al. [17, 29], building on Soft Q-learning [32]; its twin critics adopt the overestimation-bias fix introduced for off-policy actor-critic methods by TD3 [36], and the reparameterized actor gradient traces to the stochastic backpropagation of Rezende et al. [37], the same mechanism used earlier for deterministic actors in DDPG [38]. SAC uses twin Q-critics Q_{ψ_1}, Q_{ψ_2} and target networks to optimize:

$$J_{\text{ME}}(\pi) = \mathbb{E}\big[\textstyle\sum_t r(s_t, a_t) + \alpha\,\mathcal{H}\big(\pi(\cdot \mid s_t)\big)\big]\,, \tag{5}$$

with fixed temperature $\alpha = 0.1$ (not auto-tuned). The actor and critic losses are

$$J_\pi(\varphi) = \mathbb{E}_{s\sim\mathcal{D},\, a\sim\pi_\varphi}\Big[\alpha \log \pi_\varphi(a \mid s) - \min_i Q_{\psi_i}(s,a)\Big], \tag{6}$$

$$J_Q(\psi_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(Q_{\psi_i}(s,a) - y\big)^2\Big], \tag{7}$$

$$y = r + \gamma\big(\min_j Q_{\bar{\psi}_j}(s',a') - \alpha \log \pi_\varphi(a' \mid s')\big), \tag{8}$$

with soft target updates $\bar{\psi}_i \leftarrow (1-\tau_s)\bar{\psi}_i + \tau_s\psi_i$, $\tau_s = 0.005$. Per episode, the rollout's 15 transitions are added to a replay buffer $\mathcal{D}$ (capacity $= n_{\text{ep}} \times 15$) and 10 gradient updates are performed with batch size 64.

SAC is the natural off-policy counterpart to PPO along exactly the factor this study varies (Fig. 1): both optimize a stochastic policy with an actor-critic architecture, so replacing PPO's rollout-and-discard loop with SAC's replay buffer isolates the on-/off-policy factor without also changing the policy family, as switching to a deterministic actor (DDPG/TD3) would. Its replay buffer also gives the off-policy family a mechanism, buffer dilution, that PPO structurally lacks; Section 6.2 uses this to explain why on-policy and off-policy runs can behave differently immediately after a chaotic-regime trigger episode even when neither family is more failure-prone on average. We sample the buffer uniformly rather than by TD-error priority [39], so that any such difference is attributable to the on-/off-policy structure itself and not to a secondary sampling scheme.

3.7. Analytic policy gradient

The analytic gradient is the derivative of a short rollout's accumulated reward with respect to the policy parameters, obtained by unrolling the policy's *deterministic mean* action through the differentiable solver and differentiating end-to-end with reverse-mode automatic differentiation. It is a low-variance but biased estimator, so its length is capped at $H = 6$ steps and it is mixed with the model-free gradient at a fixed 50/50 weight rather than trusted alone (Fig. 3).

Backpropagating through a rollout is the classical backpropagation-through-time construction [40]; SHAC [14] and AHAC [15] apply the same short-horizon truncation to differentiable simulators, and Metz et al. [12] and Parmas et al. [23] show that such backpropagated gradients can explode and lose direction under chaotic dynamics. Over $H = 6$ steps we forward-propagate the policy's mean action (no sampling, so the whole chain stays smooth) and differentiate the negative accumulated reward:

$$\begin{aligned} \mathcal{L}_{\text{an}}(\theta) &= -\sum_{t=1}^{H} r\big(s_t, a_\mu(s_t;\theta)\big), \\ s_t &= f_{\text{sim}}(s_{t-1}, a_\mu(s_{t-1};\theta); Re), \\ g_{\text{an}} &= \nabla_\theta \mathcal{L}_{\text{an}}(\theta), \end{aligned} \tag{9}$$

computed by `jax.grad`. The final gradient mixes g_{an} with the model-free gradient $g_{\text{mf}} \in \{g_{\text{PPO}}, g_\pi\}$ at equal weight:

$$g_{\text{final}} = \beta\, g_{\text{an}} + (1-\beta)\, g_{\text{mf}}, \qquad \beta = 0.5, \tag{10}$$

the $\alpha = 0.5$ special case of the α-graded estimator of Suh et al. [25], of the total-gradient decomposition of Parmas and Sugiyama [41], and the non-adaptive limit of the GI-PPO α-policy [11]. Eq. (10) is the only structural difference between PPO and GPPO, and between SAC and GSAC.

We fix $\beta = 0.5$ and $H = 6$ rather than adapting either online, because this study asks whether analytic-gradient injection helps or hurts as the regime turns chaotic, and an adaptive scheme such as GI-PPO's α-policy or SAPO's entropy-based smoothing would let the algorithm partially opt out of the analytic term exactly when the chaotic regime makes it unreliable — confounding the effect under test. Sections 3.8 and 3.9 apply this same fixed rule to the on-policy and off-policy actor gradient respectively, so that GPPO and GSAC differ from PPO and SAC by exactly one mechanism.

3.8. GPPO: on-policy variant

GPPO adds the analytic-gradient mix of Eq. (10) to the PPO update: from the same fixed reset state, a deterministic $H = 6$-step analytic rollout runs alongside the standard stochastic on-policy rollout, and the two resulting gradients are averaged before a single Adam step (Fig. 4 and algorithm 1). Every other setting — rollout length, GAE, the clip range, network architecture, and learning rate — is identical to plain PPO (Section 3.5).

This is the same mechanism as the HydroGym paper's GPPO, reported to cut training iterations by at least 65 % over PPO on Kolmogorov flow [6]. That paper does not report its horizon length or mixing weight, so ours follow the general short-horizon-and-mix recipe of SHAC, AHAC, and GI-PPO (Section 3.7) rather than an attempt to match unpublished hyperparameters.

GPPO is the on-policy arm of the 2×2 design and the direct object of the reproduction question this paper asks: does the reported sample-efficiency gain hold, under the same fixed recipe, at a chaotic operating point where the analytic gradient's premises — smooth, low-variance backpropagation through the solver — are not expected to hold? Section 5 answers this at $Re = 45$ (yes) and $Re = 300$ (no seed-robust effect).

3.9. GSAC: off-policy variant

GSAC applies the same analytic-gradient mix to the SAC actor rather than to PPO's surrogate: after each episode's rollout and ten off-policy critic-and-actor updates, one further actor step mixes the last actor gradient, $g_{\text{mf}} = g_\pi = \nabla_\varphi J_\pi(\varphi)$ from Eq. (6), with a deterministic $H = 6$-step analytic rollout from the reset state (Fig. 5 and algorithm 2). The critics, target networks, and replay mechanics are otherwise unchanged from plain SAC (Section 3.6).

This follows SAPO's core motivation of combining an off-policy, maximum-entropy actor with an analytic gradient [16], but targets HydroGym's Kolmogorov flow rather than SAPO's rigid/deformable-body platform, and uses the same fixed 50/50 mix as GPPO (Section 3.7) rather than SAPO's entropy-based smoothing. To our knowledge this

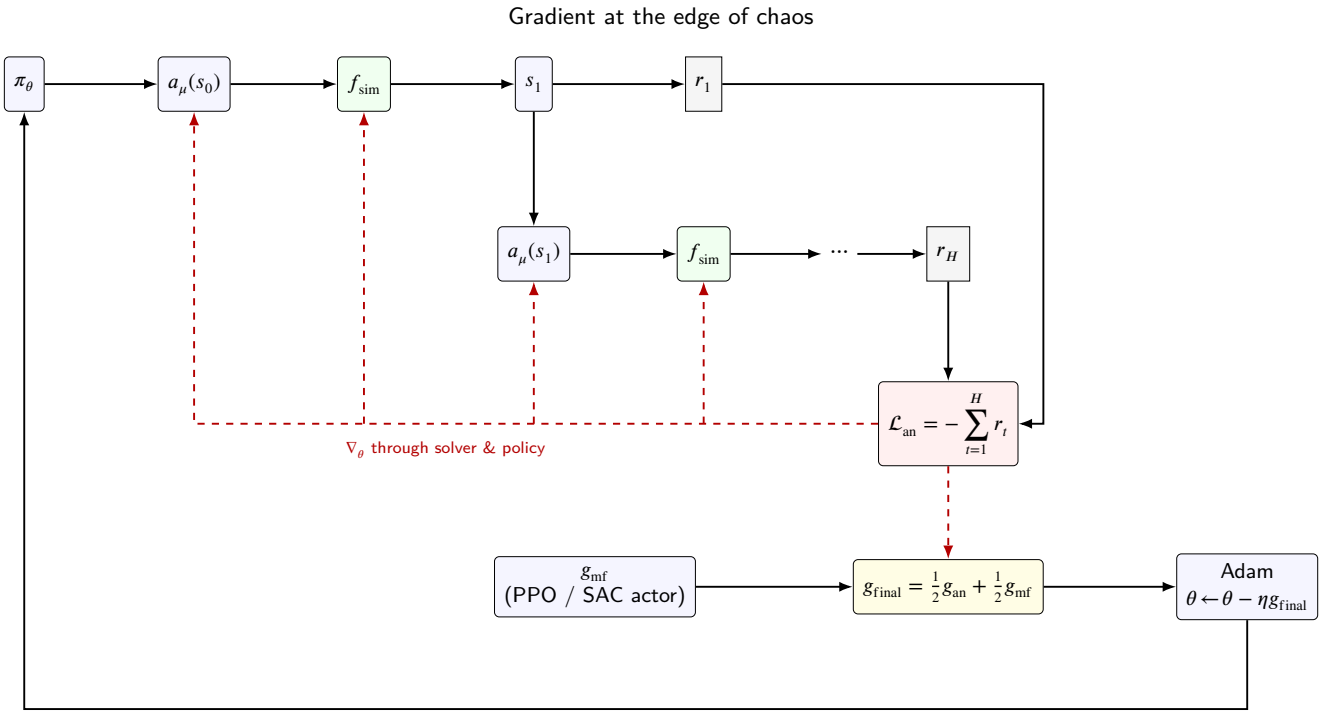


Figure 3: Analytic-gradient computation graph (Eqs. (9) and (10)). Solid arrows: forward rollout of the *deterministic mean* action through the differentiable solver for $H = 6$ steps. Dashed red arrows: reverse-mode differentiation of the accumulated reward $\mathcal{L}_{\text{an}}$ with respect to θ via `jax.grad`. The analytic gradient is averaged 50/50 with the model-free gradient and applied with Adam. Replacing the mixer with $g_{\text{final}} = g_{\text{mf}}$ recovers plain PPO / SAC.

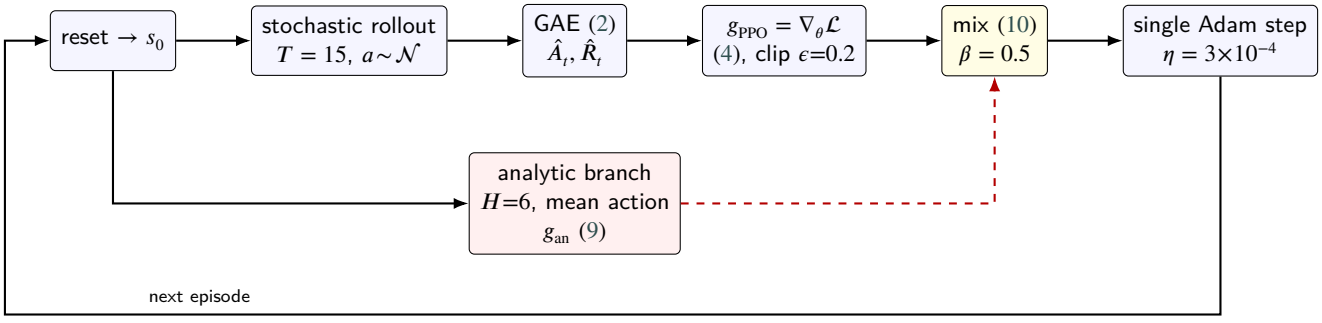


Figure 4: GPPO training loop (one episode). The analytic branch is a deterministic 6-step unroll that runs in parallel with the standard on-policy update; the two gradients are averaged before a single Adam step. Removing the red branch gives PPO.

is the first application of an off-policy, maximum-entropy analytic-gradient method to a fluid-dynamics control task.

GSAC completes the 2 × 2 design (Fig. 1) and lets this study ask its second question directly: does an off-policy replay buffer tolerate the analytic gradient's chaotic-regime instability better than an on-policy rollout, or does adding a replay buffer simply add its own failure mode? Because GPPO and GSAC share the identical analytic branch (Eq. (9)) and mixing rule (Eq. (10)), any difference in chaotic-regime robustness between them is attributable to the on-/off-policy factor rather than to a different analytic-gradient implementation.

GAE, PPO clipping, and the 50/50 analytic/model-free mix are hand-rolled in plain JAX/optax and kept identical in form across the PPO and SAC training scripts. Both scripts additionally log per-episode action amplitude (RMS and absolute max) for the trigger-episode diagnostic of Section 6.

4. Experimental setup and reproducibility

4.1. Configuration

Table 2 lists every hyperparameter. All four algorithms are trained for 500 episodes at both Reynolds numbers, matching the training budget reported by the HydroGym paper [6].

4.2. Seed protocol

Each (algorithm, Re, seed) triple is trained independently for five seeds. The JAX PRNG is fully deterministic, so a given (config, seed) is bit-for-bit reproducible — the seed = 3 GSAC/Re = 300 run was regenerated identically after its first dump was lost. Configurations are run strictly one at a time: this class of consumer GPU cannot host concurrent JAX processes without severe memory contention. Per-seed wall time for the full 8-configuration

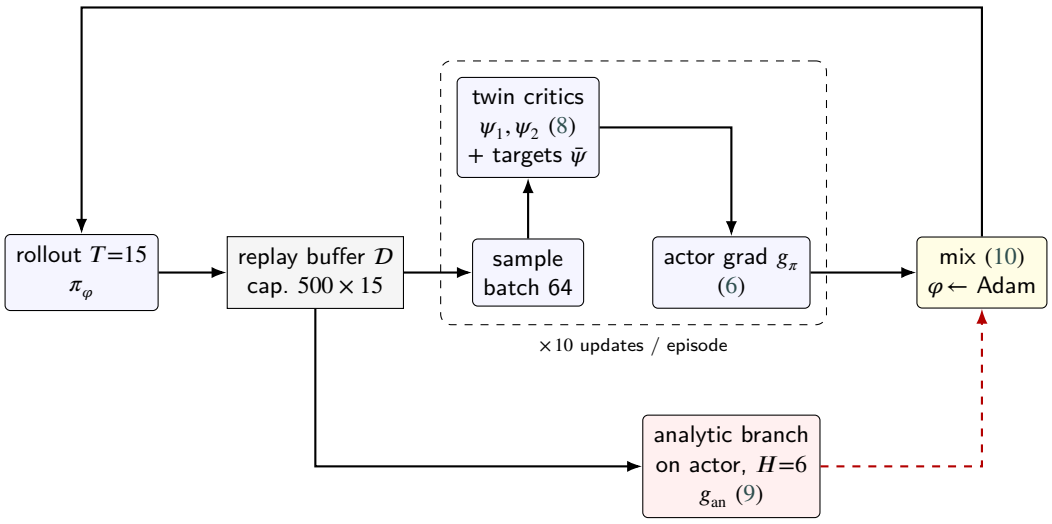


Figure 5: GSAC training loop (one episode). Ten off-policy critic+actor updates on replay batches are followed by a single analytic actor step (6-step deterministic unroll from the reset state). Removing the red branch and the mixer gives SAC.

Algorithm 1: GPPO — one training episode ($\beta = 0.5$, $H = 6$)

Input: policy θ, value ϕ, optimizer state, fixed reset state s_0

$\tau \leftarrow \text{ROLLOUT}(\pi_\theta, s_0)$ for $T = 15$ steps `// sample` $a_t \sim \mathcal{N}(a_\mu(s_t), \sigma^2)$

$\hat{A}, \hat{R} \leftarrow \text{GAE}(\tau; \gamma{=}0.99, \lambda{=}0.95)$

$g_{\text{PPO}} \leftarrow \nabla_\theta \big[-\mathcal{L}^{\text{PPO}}(\theta; \tau, \hat{A}) + 0.5\, \mathcal{L}^V(\phi; \tau, \hat{R}) - 0.001\, \mathcal{H}(\pi_\theta) \big]$

`// analytic branch: deterministic, differentiable, short horizon`

$\mathcal{L}_{\text{an}}(\theta) \leftarrow -\sum_{t=1}^{H} r\big(f_{\text{sim}}^t(s_0, a_\mu(\cdot\,; \theta))\big)$

$g_{\text{an}} \leftarrow \nabla_\theta \mathcal{L}_{\text{an}}(\theta)$ `// jax.grad through the solver`

$g \leftarrow \beta\, g_{\text{an}} + (1 - \beta)\, g_{\text{PPO}}$

$\theta \leftarrow \text{ADAM}(\theta, g;\ \eta{=}3{\times}10^{-4})$ `// single full-batch step`

Output: θ, ϕ;
logs $\big(\|g_{\text{an}}\|,\ \text{RMS}|a_t|,\ \max|a_t|,\ \sum_t r_t\big)$

PPO is this algorithm with the analytic branch (the $\mathcal{L}_{\text{an}}$, g_{an}, and mixing steps) removed, so that $g \leftarrow g_{\text{PPO}}$.

Algorithm 2: GSAC — one training episode ($\beta = 0.5$, $H = 6$)

Input: actor φ, critics ψ_1, ψ_2, targets $\bar{\psi}_1, \bar{\psi}_2$, replay buffer $\mathcal{D}$, reset state s_0

$\tau \leftarrow \text{ROLLOUT}(\pi_\varphi, s_0)$ for $T = 15$ steps; $\mathcal{D} \leftarrow \mathcal{D} \cup \tau$

for $k = 1 \ldots 10$ **do**

  $B \leftarrow \text{SAMPLE}(\mathcal{D}, 64)$

  for $i \in \{1, 2\}$ **do**

    $\psi_i \leftarrow \text{ADAM}\big(\psi_i, \nabla_{\psi_i} J_Q(\psi_i; B)\big)$

  $g_\pi \leftarrow \nabla_\varphi\, \mathbb{E}_{s \in B}\big[\alpha \log \pi_\varphi(a \mid s) - \min_i Q_{\psi_i}(s, a)\big],\ a \sim \pi_\varphi(\cdot \mid s),\ \alpha = 0.1$

  $\varphi \leftarrow \text{ADAM}(\varphi, g_\pi)$

  $\bar{\psi}_i \leftarrow (1 - \tau_s)\bar{\psi}_i + \tau_s \psi_i, \quad \tau_s = 0.005$

`// analytic branch on the actor, after the SAC updates`

$g_{\text{an}} \leftarrow \nabla_\varphi \big[-\sum_{t=1}^{H} r\big(f_{\text{sim}}^t(s_0, a_\mu(\cdot\,; \varphi))\big)\big]$

$\varphi \leftarrow \text{ADAM}\big(\varphi,\ \beta\, g_{\text{an}} + (1 - \beta)\, g_\pi\big)$ `// $g_\pi$ from the last inner step`

Output: φ, ψ;
logs $\big(\|g_{\text{an}}\|,\ \text{RMS}|a_t|,\ \max|a_t|,\ \sum_t r_t\big)$

SAC is this algorithm with the analytic branch (the final g_{an} and mixing steps) removed.

sweep is ∼6–8.5 h (seed = 4: ∼6.2 h; PPO ∼28 min/config, GPPO ∼63 min, SAC ∼28 min, GSAC ∼60 min).

4.3. Reproducibility package

Following the ML reproducibility checklist [42], the release will include the experiment code as a standalone repository separated from the upstream HydroGym clone; a pinned environment specification (122 packages), a portable environment-rebuild script, and the parametrized sweep driver; every per-configuration result dump — episode rewards, analytic-gradient norms, action-amplitude logs, and wall times — together with the driver logs; and a Zenodo DOI accompanied by "Code availability" and "Data availability" statements.

4.4. Analysis of the chaotic-regime outcome

The outcome variable at $Re = 300$ is the *first-NaN episode* — the training episode at which the flow simulation first returns a non-finite reward. This is not a short-circuit: training continues to run all 500 episodes regardless. The four algorithms are compared pairwise on their five paired per-seed differences using an exact sign-flip permutation test

Table 2
Full configuration. Identical across all four algorithms except where an algorithm-specific row applies.

Parameter	Value
Grid	64×64 (4096 DOF)
Solver	pseudo-spectral IMEX-RK4CN (real `hydrogym.jax`)
Re (regular / chaotic)	45 / 300
Episodes / steps per episode	500 / 15
Environment dt / `action_time`	0.001 / 1.0
Reward	$\alpha_{\text{reward}} = 1$ (flow energy + control cost)
Analytic horizon (GPPO/GSAC)	$H = 6$ steps, mean (deterministic) action
Gradient mixing	$0.5\cdot$ model-free + $0.5\cdot$ analytic
Optimizer / learning rate	Adam / 3×10^{-4} (actor and critic)
Policy / value / critic network	2 hidden layers $\times$ 64 units, tanh
Action	PPO: $a_\mu = 0.5\tanh(\cdot)$, sampled, not re-clipped; SAC: squashed tanh
PPO	γ=0.99, λ=0.95, clip ϵ=0.2, entropy coeff. 0.001, 1 full-batch step / episode
SAC	γ=0.99, α=0.1 (fixed), τ_s=0.005, 10 updates / episode, batch 64, buffer = 500×15
Seeds	$n = 5$ (seed = 0–4)
Hardware	RTX 5060 laptop GPU (8 GB), JAX 0.10.2 + CUDA 12

(all $2^5 = 32$ relabelings, two-sided) and a bootstrap 95 % CI for the median difference (20 000 resamples). Omnibus tests are Kruskal–Wallis (unblocked) and Friedman (blocked by seed). The pre-registered decision rule (Section 1): report "no seed-robust ordering" unless GSAC beats every other algorithm at $p < 0.05$ one-sided.

5. Results

5.1. Regular regime ($Re = 45$): sample efficiency

At $Re = 45$, GPPO reaches PPO's best 500-episode smoothed reward in substantially fewer episodes in 4 of 5 seeds (seed = 0: 63 % fewer; seed = 1: ∼78 %; seed = 3: ∼84 %; seed = 4: ∼70 %), and by a smaller margin at seed = 2 (∼26 %). The *direction* of GPPO's advantage over PPO holds in all five seeds, and the seed = 0 value (63 %) is close to the paper's reported 65 % [6] — evidence that our reimplementation captures the paper's mechanism. Fig. 6 shows the seed-averaged learning curves.

Two $Re = 45$ findings are seed-stable (Fig. 7): (i) the off-policy family reaches clearly better final reward than the on-policy family in every seed — its replay-buffer structure is already far more sample-efficient, so no GPPO-style acceleration analysis applies to it; and (ii) GSAC does not exceed SAC's final performance in any seed — the analytic gradient does not help the off-policy method in the regular regime.

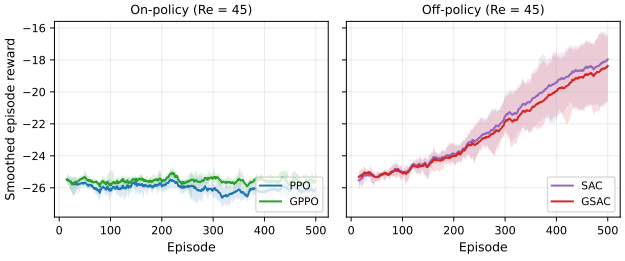


Figure 6: Regular regime ($Re = 45$). Seed-averaged smoothed episode reward (15-episode window); shaded band is the seed-wise min–max envelope over the five seeds. The off-policy family (right) converges to a markedly higher reward level than the on-policy family (left); within each family the analytic-gradient variant tracks its base algorithm closely.

Table 3 gives the exact per-seed values underlying Fig. 7. seed = 2 trained to a worse level across all four algorithms, but the *relative* ordering is unchanged.

5.2. Chaotic regime ($Re = 300$): the breaking point has no stable ordering

In the chaotic regime all four algorithms have a breaking point — none completes 500 episodes stably; all eventually diverge (NaN) (Fig. 8). The episode at which the break first occurs varies by algorithm, but far more so *by seed*: the ranking permutes across the five seeds (Fig. 9, Table 4).

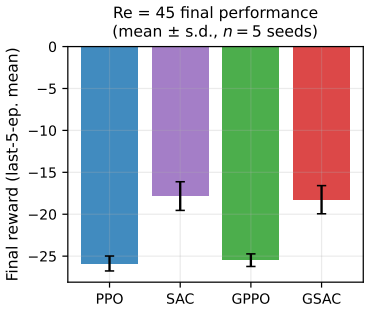


Figure 7: Final reward at $Re = 45$ (last-5-episode mean, mean ± s.d. over five seeds). SAC/GSAC clearly outperform PPO/GPPO; GSAC does not exceed SAC.

Table 3
Final performance at $Re = 45$, last-5-episode mean reward (seed = 0 / 1 / 2 / 3 / 4).

Algorithm	Final reward
PPO	−25.1 / −25.2 / −27.2 / −26.4 / −25.6
GPPO	−25.4 / −24.6 / −26.7 / −25.4 / −25.4
SAC	−16.7 / −16.9 / −18.9 / −16.3 / −20.3
GSAC	−18.0 / −17.1 / −20.4 / −16.3 / −19.5

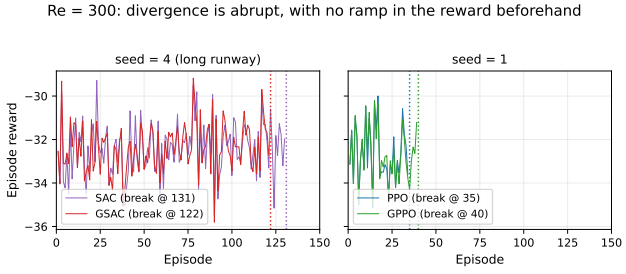


Figure 8: Example $Re = 300$ training curves. Left: the two long-runway seed = 4 off-policy runs (SAC breaks at episode 131, GSAC at 122). Right: the seed = 1 on-policy runs (PPO at 35, GPPO at 40). In every case the reward is stationary until a single episode at which it collapses; the dotted line marks the first-NaN episode.

The episode at which this collapse occurs is summarized across all seeds and algorithms in Fig. 9.

Table 4 lists the exact per-seed episode counts underlying this figure, together with the per-seed most-robust algorithm.

At $n = 2$ (the conference version) two claims survived: analytic-gradient injection delays the break (GPPO > PPO, GSAC > SAC) in direction, and GSAC is the most robust of the four. At $n = 5$ neither is a statistically separable ordering.

Analytic-gradient injection does not delay the on-policy break: GPPO > PPO holds at seeds 0, 1, 4 (83 vs 12; 40 vs 35; 16 vs 11) but reverses hard at seed = 2 (32 vs 117) and seed = 3 (6 vs 14, GPPO breaking first of all four). The paired per-seed difference GPPO−PPO is $\{+71, +5, -85, -8, +5\}$, mean $\mathbf{-2.4}$ episodes, sign-flip permutation $p = \mathbf{0.81}$.

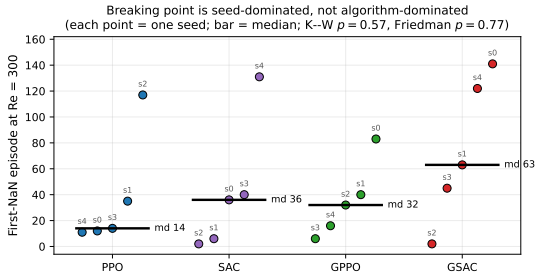


Figure 9: First-NaN episode at $Re = 300$: each point is one seed, the horizontal bar is the per-algorithm median. Every algorithm's range spans more than an order of magnitude, the medians overlap heavily, and no omnibus test detects an algorithm effect (Kruskal–Wallis $p = 0.57$, Friedman $p = 0.77$). GSAC has the highest median but is tied-last at seed = 2.

GSAC leads by median but not significantly: its breaking point (median 63, mean 74.6) is the highest of the four; it is the most robust algorithm at seeds 0, 1, 3, second at seed = 4, and tied-last at seed = 2 (the early-init failure mode of Section 6.3 — the flow diverges before learning begins and bare SAC fails identically at the same episode). But the spread is enormous: GSAC−SAC per seed is $\{+105, +57, 0, +5, -9\}$, $p = 0.38$; GSAC−GPPO is $\{+58, +23, -30, +39, +106\}$, $p = 0.19$; GSAC−PPO $p = 0.38$.

What does hold across all five seeds are three seed-stable statements. Every algorithm eventually diverges at $Re = 300$: analytic-gradient injection never prevents the break, it at most postpones it. The break is never preceded by a visible precursor in the reward or gradient signal (Section 6.1). And the breaking point is dominated by the seed — within-algorithm variance across seeds (s.d. 30–57 episodes) swamps the between-algorithm differences in mean (spread of means 35–75). The "clean multiplicative" structure reported at $n = 1$ and the "GSAC most robust" claim that survived $n = 2$ are both artifacts of small n. GSAC carries a non-significant robustness lean — highest median, most robust in 3/5 — reported as a candidate effect, not a claim.

5.3. Significance test ($n = 5$)

Table 5 gives the full pairwise comparison.

Omnibus: Kruskal–Wallis $H = 2.00$, $p = 0.57$; Friedman $\chi^2 = 1.80$, $p = 0.77$. No pairwise or omnibus comparison reaches $p < 0.05$. Per the pre-registered decision rule (Section 1), the finding stands as no statistically separable ordering of chaotic-regime training stability by algorithm. GSAC's non-significant lean (highest median and mean, most robust in 3/5 seeds, all pairwise $p \in [0.19, 0.38]$) is reported as a candidate effect for a future larger-n study.

5.4. Analytic-gradient norm

Fig. 10 plots the analytic-gradient norm $\|g_{\text{an}}\|$ over training, up to the break, for GPPO and GSAC in both regimes. Contrary to the pre-registered hypothesis, $\|g_{\text{an}}\|$ does *not* blow up before the break at $Re = 300$: it fluctuates in the same ∼20–70 band as at $Re = 45$ and shows no upward trend approaching the divergence episode. The instability

Table 4
First-NaN episode at $Re = 300$ across the five seeds, with per-algorithm median and mean. "Most robust" = latest break in that seed.

Algorithm	seed= 0	seed= 1	seed= 2	seed= 3	seed= 4	median	mean
PPO	12	35	117	14	11	14	37.8
SAC	36	6	2	40	131	36	43.0
GPPO	83	40	32	6	16	32	35.4
GSAC	141	63	2	45	122	63	74.6
most robust	GSAC	GSAC	PPO	GSAC	SAC	—	—

Table 5
Pairwise breaking-point comparison at $Re = 300$, $n = 5$. Exact sign-flip permutation p (two-sided); bootstrap 95 % CI for the median difference.

Comparison	per-seed diff (s0–s4)	mean	median	95 % CI (median)	p
GPPO − PPO	+71, +5, −85, −8, +5	−2.4	+5	[−85, +71]	0.81
GSAC − SAC	+105, +57, 0, +5, −9	+31.6	+5	[−9, +105]	0.38
GSAC − GPPO	+58, +23, −30, +39, +106	+39.2	+39	[−30, +106]	0.19
GSAC − PPO	+129, +28, −115, +31, +111	+36.8	+31	[−115, +129]	0.38
SAC − PPO	+24, −29, −115, +26, +120	+5.2	+24	[−115, +120]	0.88
GPPO − SAC	+47, +34, +30, −34, −115	−7.6	+30	[−115, +47]	0.88

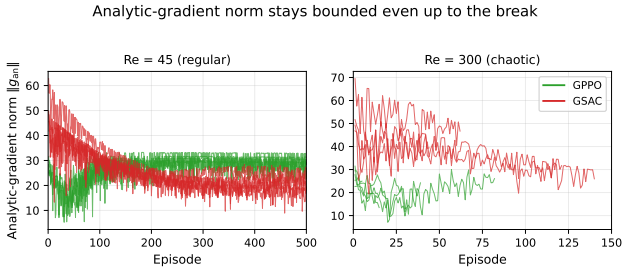


Figure 10: Analytic-gradient norm $\|g_{\text{an}}\|$ per episode (all five seeds overlaid), truncated at each run's first-NaN episode. Left: $Re = 45$. Right: $Re = 300$. The norm stays bounded and trend-free right up to the break; the chaotic-regime divergence is not preceded by a gradient explosion.

that ends training is therefore not visible in the gradient magnitude — a point developed mechanistically in Section 6.

6. Mechanism of the break

Sections 6.1 and 6.2 are direct inspections of raw training curves; Section 6.3 uses the per-episode action-amplitude log (Sections 3.9 and 4). Fig. 11 summarizes the two surviving mechanistic results.

6.1. The break is sudden and precursor-free

For all four algorithms, in every seed with a mid-training break (seeds 0, 1, 3, 4), the mean and standard deviation of the reward are stable over the 40-episode window immediately before the break — no ramp, no gradual degradation (Table 6). The longest-runway confirmations come from seed = 4, where `sac_high` (break at 131) and `gsac_high` (break at 122) each have a full 40-episode pre-break window flat to within noise: −32.29 → −32.32 (s.d. 1.09) and −32.29 → −32.08 (s.d. 1.18) respectively (Fig. 11a).

GPPO/GSAC analytic-gradient norms over the 15 episodes before the break are similarly flat (Fig. 10). The break is therefore not the policy *gradually* driving the solver past its limit, but a sudden, stochastic event — a single episode's randomly sampled action sequence, combined with the current policy, pushing the flow past the solver's $Re \approx 350$–380 numerical-stability limit (Section 3.3) — consistent with the butterfly-effect trigger expected from Metz et al. [12] and the Re-shrunk predictability horizon of Boffetta and Musacchio [13].

6.2. On-policy vs. off-policy after the trigger: buffer dilution

In on-policy PPO/GPPO the trigger episode's rollout is used in that same episode's update, so trigger and permanent collapse always coincide — no escape, in any of the 10 on-policy runs (Table 7). Off-policy training *could* in principle dilute the poisoned transitions in its replay buffer. Whether it does depends on how full the buffer is at the trigger. Let a run trigger at episode e, so the buffer holds $\approx 15e$ transitions of which ≈ 15 are poisoned; the probability that *none* of the $10 \times 64 = 640$ sampled transitions across the 10 post-trigger updates is poisoned is

$$P_{\text{survive}}(e) \approx \left(1 - \frac{15}{15\,e}\right)^{640} = \left(1 - \frac{1}{e}\right)^{640}. \quad (11)$$

Table 6
Pre-break reward statistics (40-episode window where the runway allows, else the full pre-break run; seed = 0 / 1 / 4).

Algorithm	First-half mean	Second-half mean	Std. dev.
PPO	−32.75 / −32.28 / −32.36	−32.77 / −33.05 / −32.39	1.35 / 1.32 / 0.91
SAC	−33.31 / −32.76 / −32.29	−32.50 / −32.79 / −32.32	1.30 / 1.39 / 1.09
GPPO	−32.61 / −32.38 / −32.60	−32.98 / −33.01 / −33.05	1.10 / 1.24 / 1.34
GSAC	−32.49 / −32.19 / −32.29	−32.46 / −32.08 / −32.08	1.32 / 1.16 / 1.18

Table 7
Trigger episode vs. recovery. "Recovered" = intermediate episodes with a valid (non-NaN) reward between trigger and permanent collapse. Across all 20 $Re = 300$ runs exactly one shows a recovery window: seed = 0 GSAC (2 episodes).

Algorithm	Trigger (s0 / s1 / s2 / s3 / s4)	Recovered
PPO	12 / 35 / 117 / 14 / 11	0 / 0 / 0 / 0 / 0
SAC	36 / 6 / 2 / 40 / 131	0 / 0 / 0 / 0 / 0
GPPO	83 / 40 / 32 / 6 / 16	0 / 0 / 0 / 0 / 0
GSAC	141 / 63 / 2 / 45 / 122	**2** / 0 / 0 / 0 / 0

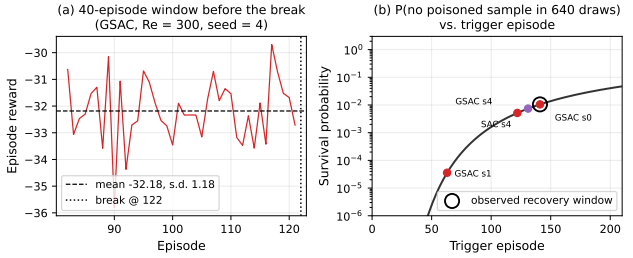


Figure 11: Two surviving mechanistic results. (a) The 40-episode reward window before the break (GSAC, $Re = 300$, seed = 4): flat to within noise, then an abrupt divergence. (b) The buffer-dilution survival probability of Eq. (11) vs. trigger episode; markers are the actual off-policy runs, and the single observed recovery window (circled, seed = 0 GSAC, trigger 141) is the one case where P_{survive} is ∼1%.

Eq. (11) is negligible unless $e \gtrsim 120$ (Fig. 11b): a run triggering at episode 36 has $P_{\text{survive}} \approx 0$, whereas one triggering at episode 141 has $P_{\text{survive}} \approx 1\%$. Only GSAC triggers late enough (seeds 0 and 4, episodes 141 and 122) for this to be non-negligible.

Fig. 11b plots Eq. (11) against the actual trigger episodes of the ten off-policy runs in Table 7, with the single observed recovery window circled.

Eq. (11) gives a non-zero survival probability only for a GSAC run that triggers past ∼episode 120. Two of the five GSAC runs meet this (seed = 0 at 141, seed = 4 at 122); the model predicts a recovery window is *possible but rare* (∼1%) for these and effectively impossible for the other 18 runs. Observed: one window, in seed = 0 GSAC. This is consistent with the model, but with one positive across 20 runs the mechanism is suggestive, not confirmed; a controlled buffer-size sweep at a fixed trigger is needed to establish the ∼1% rate.

6.3. Divergence before learning: the early-init failure mode

The seed = 2 sweep exposes a second, qualitatively different failure mode. In seed = 2 at $Re = 300$, both SAC and GSAC break at episode 2 with an *identical* episode-1 trajectory: same reward (−33.68), same action RMS (0.308), same action abs-max (0.489). This identity is expected — SAC and GSAC share the seed's initialization and diverge only after episode 1, when GSAC applies its extra analytic step — yet both reach NaN on episode 2.

The logs rule out the Section 6.2 mechanism and an action-explosion mechanism alike. GSAC's episode-1 analytic-gradient norm is 54, squarely in the normal range (seed = 0: 49, seed = 1: 69 over their first ten episodes), so the analytic step was not anomalous; and the episode-2 rollout's action RMS is 0.307 with abs-max 0.485, indistinguishable from episode 1, so the policy is not driving the flow with large control. What remains is that seed = 2's randomly initialized policy, composed with the $Re = 300$ base flow, integrates to numerical divergence within two episodes — before any actor-learning algorithm has a foothold. Bare SAC diverges identically, so this is a property of where the random initialization lands relative to the solver's stability boundary (Section 3.3), not of analytic-gradient injection. GSAC's long runways in seeds 0, 1, 4 (141, 63, 122 episodes) require several non-diverged episodes for the analytic gradient to accumulate a stabilizing effect; seed = 2 provides no such runway.

This mode is rare: 1 of 5 seeds. Seeds 3 and 4 both break mid-training, so the episode-2 collapse is not the typical $Re = 300$ behaviour — it is a low-probability tail of the same sudden, precursor-free, stochastic trigger of Section 6.1, occurring at episode 2 instead of 45–141. It still reframes GSAC as highest ceiling, no reliable floor, and shows the trigger-episode distribution reaches all the way down to episode 2.

Section 6.2 is a plausibility calculation for the post-trigger lock-in; Section 6.3 shows *when* the trigger can occur. Neither explains *why* a given (seed, init) triggers at a given episode. The action-amplitude log now in place is the instrument for this; a dedicated short-horizon re-run of seeds 0–1 logging the per-step action norm is the next diagnostic (Section 8).

7. Discussion

This work extends the HydroGym paper's GPPO-vs-PPO result on Kolmogorov flow [6] in two directions, and the second produces a seed-dominance result that is itself the main finding.

At the regular operating point ($Re = 45$) the paper's core claim holds: GPPO reaches PPO's best reward in substantially fewer episodes in 4 of 5 seeds (≈63 % to 84 %, seed = 0's 63 % closest to the reported 65 %), the direction is seed-stable across all five, and the off-policy family reaches better final reward than the on-policy family in every seed. GSAC never beats SAC here — the analytic gradient does not help the off-policy method when the flow is not chaotic. This is consistent with the SAPO [16] reading that the analytic gradient earns its keep by *stabilizing* a hard landscape rather than by accelerating an easy one; at $Re = 45$ SAC's replay buffer already solves the task.

At the chaotic operating point ($Re = 300$), by contrast, no separable ordering survives: the conference version, at $n = 2$, reported that GSAC is the most robust of the four and that analytic-gradient injection delays the break in direction. Over five seeds the second claim does not hold at all (GPPO−PPO mean −2.4 episodes, sign-flip $p = 0.81$) and the first survives only as a non-significant lean (every pairwise permutation $p \geq 0.19$; Kruskal–Wallis $p = 0.57$, Friedman $p = 0.77$). Each algorithm's first-NaN episode spans more than an order of magnitude; PPO, first to break at seed = 0, is the most robust of the four at seed = 2; at seed = 2 both off-policy methods collapse at episode 2. The "clean multiplicative" $n = 1$ story and the "GSAC most robust" $n = 2$ story are both small-n artifacts. The finding that replaces them: in the chaotic regime the breaking point is dominated by the random path of training, not by the algorithm — within-algorithm cross-seed s.d. (30–57 episodes) swamps the spread of between-algorithm means (35–75). Analytic-gradient injection, off-policy structure, and their combination fail to impose a statistically separable ordering on when a differentiable chaotic-flow simulation diverges under RL training. This is a concrete, quantified instance of the chaos-induced gradient instability of Metz et al. [12] and the Re-shrunk predictability horizon of Boffetta–Musacchio [13]: at this problem scale the instability is not merely large, it is *irreducibly stochastic*. It also matches what RPO [18] targets — the training instability of BPTT-through-a-surrogate — and suggests that the RPO/GI-PPO line of adaptive weighting, rather than a fixed mix, is the right tool for this regime.

Two mechanistic results do survive, both algorithm-design-independent. The first is buffer dilution (Section 6.2): a post-trigger survival window has non-zero probability only when the poisoned fraction of the replay buffer at the trigger is ∼1% or below — that is, only for GSAC runs that trigger past ∼episode 120 — and exactly one of the 20 $Re = 300$ runs shows such a window, in precisely that case, so the mechanism is suggestive rather than confirmed. The second is the early-init failure mode (Section 6.3): when the randomly initialized policy composed with the $Re = 300$ base flow already sits near the solver's stability boundary, divergence occurs within about two episodes, before any actor-learning algorithm has a foothold; bare SAC and GSAC fail identically here, with normal control amplitude and a normal analytic-gradient norm at the point of divergence. Together these explain why GSAC has the highest ceiling of the four (141, 63, and 122 episodes when it is given a runway) but no reliable floor (episode 2 at seed = 2).

For a practitioner adding an analytic through-simulator gradient to an RL controller for a near-turbulent flow: (i) expect no reliable improvement in training stability from the gradient alone, and budget for divergence at an unpredictable episode; (ii) an off-policy buffer buys a recovery chance only if the divergence happens late, when the buffer is large — it is not a general safety net; and (iii) the useful engineering artifacts are the diagnostics (first-NaN-episode distribution, pre-break flatness check, buffer-dilution estimate), not a ranking of algorithms. None of this refutes the HydroGym paper's core claim that GPPO helps at its tested operating point; it shows the benefit does not carry a seed-robust ordering once the regime is chaotic.

Several caveats bound these conclusions. The study uses five seeds: this is enough to falsify the $n = 2$ orderings and to support the negative claim, but GSAC's non-significant median lean ($p \approx 0.19$–0.38) would need a substantially larger seed budget to confirm or reject — though, because the finding is one of seed-dominance, additional seeds are expected to strengthen it rather than overturn it. The chaotic regime is $Re = 300$ rather than the originally intended $Re = 450$, forced by the solver's $Re \approx 350$–380 stability limit at this grid and time step; a milder chaotic regime might admit a more stable ordering, which the planned Reynolds sweep $\{45, 100, 200, 300\}$ is designed to test. The GPPO/GSAC mixing ratio, horizon, and remaining hyperparameters are our own choices, since the HydroGym paper does not publish them, so the 63 % versus 65 % agreement is a strong corroborating signal but not a claim of identical implementation. The PPO update is deliberately minimal (one full-batch step per episode), which makes "PPO" here weaker than a tuned implementation; the comparison is therefore relative and budget-matched across all four algorithms rather than absolute. The episodes (15 RL steps) and the analytic horizon (6 steps) are both short and may be insufficient to observe the full course of RL convergence or of chaotic divergence, and the 50/50 analytic/model-free mix is held fixed — the adaptive α-graded estimator [25], AHAC's [15] adaptive horizon, and SAPO's [16] entropy-based smoothing are not tested. Finally, the two mechanistic results rest on thin evidence: the buffer-dilution calculation of Section 6.2 is supported by a single recovery window in 20 runs, and the early-init failure of Section 6.3 by a single seed; both are real but rare, and neither is established as a rate.

8. Conclusion and future work

Analytic-gradient injection does not eliminate numerical instability in the chaotic regime and, across independent training seeds, does not reliably delay the break either. On-policy injection has no measurable effect on the breaking point, and while the gradient-enhanced off-policy method GSAC has the highest median breaking point and is the most robust of the four in most seeds, no pairwise or omnibus test reaches significance; the ordering suggested by a small seed budget does not survive a larger one. The reproducible statement is one of seed-dominance: in the chaotic regime the episode at which training first diverges is dominated by the random path of training, not by the algorithm. What survives is mechanistic: the break is sudden and precursor-free; the replay buffer's size at the trigger sets whether a brief post-trigger survival window can appear; and in a rare mode the flow diverges within a couple of episodes regardless of algorithm because the random initial policy already sits near the solver's stability boundary. On this evidence GSAC has the highest ceiling of the four but no reliable floor.

Five directions follow directly. First, extend well beyond five seeds to test GSAC's non-significant median lean; the seed-dominance of the breaking point means a decisive test needs many seeds. Second, resolve the trigger-episode cause with an action-amplitude-logged short-horizon re-run of seeds 0–1 — is the trigger a single large sampled action or a resonant sequence? Third, test whether GSAC's floor can be raised (not its ceiling) with cheap, principled guards: analytic-gradient norm clipping, a non-finite-gradient fallback to pure SAC, a few-episode analytic warmup, and an adaptive mixing weight in the style of the GI-PPO α-policy or of AHAC / SAPO. Fourth, run a Reynolds sweep $\{45, 100, 200, 300\}$ to locate the Reynolds number at which any algorithm ordering disappears, framed with the predictability-horizon scaling of Boffetta and Musacchio. Fifth, reconcile against the official HydroGym GPPO implementation and a stronger PPO baseline with epochs and minibatching, and extend the comparison to a second environment.

Acknowledgment

The authors thank Sakarya University's National Technology Workshop Artificial Intelligence Center for the computational resources used in this study.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) in order to improve the language and readability of the manuscript and to assist with code and figure generation. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare no competing interests.

Data and code availability

The experiment code, pinned environment specification, sweep drivers, and every per-configuration result dump (episode rewards, analytic-gradient norms, action-amplitude logs, wall times) are publicly available at https://github.com/GulogluHaktan/hydrogym-work. The release v1.0.0 is permanently archived on Zenodo (https://doi.org/10.5281/zenodo.23452182).

CRediT authorship contribution statement

Cem Özkurt: Conceptualization, Methodology, Supervision, Project administration.
Muhammed Emin Onay: Data curation, Visualization, Investigation, Writing – review & editing.
Eyüp Altuğ Tunç: Software, Validation, Investigation, Writing – review & editing.
Ahmet Kutay Küçükler: Formal analysis, Resources, Validation, Writing – review & editing.
Haktan Yusuf Güloğlu: Software, Methodology, Writing – original draft, Writing – review & editing.

References

[1] S. L. Brunton, B. R. Noack, P. Koumoutsakos, Machine learning for fluid mechanics, Annual Review of Fluid Mechanics 52 (2020) 477–508. doi:10.1146/annurev-fluid-010719-060214.

[2] R. Vinuesa, S. L. Brunton, Enhancing computational fluid dynamics with machine learning, Nature Computational Science 2 (2022) 358–366. doi:10.1038/s43588-022-00264-7.

[3] J. Rabault, M. Kuchta, A. Jensen, U. Réglade, N. Cerardi, Artificial neural networks trained through deep reinforcement learning discover control strategies for active flow control, Journal of Fluid Mechanics 865 (2019) 281–302. doi:10.1017/jfm.2019.62.

[4] B. Font, F. Alcántara-Ávila, J. Rabault, R. Vinuesa, O. Lehmkuhl, Deep reinforcement learning for active flow control in a turbulent separation bubble, Nature Communications (2025). doi:10.1038/s41467-025-56408-6.

[5] J. Zhang, C. Xia, X. Jiang, I. Fumarola, G. Rigas, Real-time reinforcement learning for turbulent state-dependent control in a bluff-body wake, Communications Engineering (2026). doi:10.1038/s44172-026-00747-9.

[6] C. Lagemann, S. Mokbel, M. Gondrum, M. Rüttgers, Y. Wang, P. Suárez, L. Paehler, D. A. Bezgin, A. B. Buhendwa, J. L. Callaham, S. Ahnert, N. Zolman, X. Shao, J.-C. Loiseau, N. Adams, M. Meinke, W. Schröder, K. Lagemann, E. Lagemann, R. Vinuesa, S. L. Brunton, The HydroGym reinforcement learning platform for fluid dynamics, Nature 657 (8131) (2026) 369–376. doi:10.1038/s41586-026-10917-6.

[7] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, S. Hoyer, Machine learning–accelerated computational fluid dynamics (JAX-CFD), Proceedings of the National Academy of Sciences 118 (21) (2021). doi:10.1073/pnas.2101784118.

[8] D. A. Bezgin, A. B. Buhendwa, N. A. Adams, JAX-Fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows, Computer Physics Communications 282 (2023) 108527. doi:10.1016/j.cpc.2022.108527.

[9] D. A. Bezgin, A. B. Buhendwa, N. A. Adams, JAX-Fluids 2.0: Towards HPC for differentiable CFD of compressible two-phase flows, Computer Physics Communications 308 (2025) 109433. doi:10.1016/j.cpc.2024.109433.

[10] J. Becktepe, A. Franz, N. Thuerey, S. Peitz, Plug-and-play benchmarking of reinforcement learning algorithms for large-scale flow control (FluidGym), arXiv preprint arXiv:2601.15015 (2026).

[11] S. Son, L. Y. Zheng, R. Sullivan, Y.-L. Qiao, M. C. Lin, Gradient informed proximal policy optimization, in: Advances in Neural Information Processing Systems (NeurIPS), 2023, arXiv:2312.08710.

[12] L. Metz, C. D. Freeman, S. S. Schoenholz, T. Kachman, Gradients are not all you need, arXiv preprint arXiv:2111.05803 (2021).

[13] G. Boffetta, S. Musacchio, Chaos and predictability of homogeneous-isotropic turbulence, Physical Review Letters 119 (5) (2017) 054102. doi:10.1103/PhysRevLett.119.054102.

[14] J. Xu, V. Makoviychuk, Y. Narang, F. Ramos, W. Matusik, A. Garg, M. Macklin, Accelerated policy learning with parallel differentiable simulation (SHAC), in: Proc. International Conference on Learning Representations (ICLR), 2022, arXiv:2204.07137.

[15] I. Georgiev, K. Srinivasan, J. Xu, E. Heiden, A. Garg, Adaptive horizon actor-critic for policy learning in contact-rich differentiable simulation (AHAC), in: Proc. International Conference on Machine Learning (ICML), 2024, arXiv:2405.17784.

[16] E. Xing, V. Luk, J. Oh, Stabilizing reinforcement learning in differentiable multiphysics simulation (SAPO), in: Proc. International Conference on Learning Representations (ICLR), Spotlight, 2025, arXiv:2412.12089.

[17] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: Proc. International Conference on Machine Learning (ICML), 2018, arXiv:1801.01290.

[18] H. Zhong, X. Wang, Z. Li, L. Huang, Reparameterization proximal policy optimization, arXiv preprint arXiv:2508.06214 (2025).

[19] M. A. Bucci, O. Semeraro, A. Allauzen, G. Wisniewski, L. Cordier, L. Mathelin, Control of chaotic systems by deep reinforcement learning, Proceedings of the Royal Society A 475 (2231) (2019) 20190351. doi:10.1098/rspa.2019.0351.

[20] F. Ren, J. Rabault, H. Tang, Applying deep reinforcement learning to active flow control in weakly turbulent conditions, Physics of Fluids 33 (3) (2021) 037121. doi:10.1063/5.0037371.

[21] S. Verma, G. Novati, P. Koumoutsakos, Efficient collective swimming by harnessing vortices through deep reinforcement learning, Proceedings of the National Academy of Sciences 115 (23) (2018) 5849–5854. doi:10.1073/pnas.1800923115.

[22] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, P. Stone, Deep reinforcement learning for robotics: A survey of real-world successes, Annual Review of Control, Robotics, and Autonomous Systems (2025).

[23] P. Parmas, C. E. Rasmussen, J. Peters, K. Doya, PIPPS: Flexible model-based policy search robust to the curse of chaos, in: Proc. International Conference on Machine Learning (ICML), 2018, pp. 4065–4074, arXiv:1902.01240.

[24] A. Wolf, J. B. Swift, H. L. Swinney, J. A. Vastano, Determining lyapunov exponents from a time series, Physica D: Nonlinear Phenomena 16 (3) (1985) 285–317. doi:10.1016/0167-2789(85)90011-9.

[25] H. J. T. Suh, M. Simchowitz, K. Zhang, R. Tedrake, Do differentiable simulators give better policy gradients?, in: Proc. International Conference on Machine Learning (ICML), 2022, arXiv:2202.00817.

[26] K. Onoda, P. Parmas, M. Yaguchi, Y. Matsuo, Does "do differentiable simulators give better policy gradients?" give better policy gradients?, arXiv preprint arXiv:2604.18161 (2026).

[27] F. Li, F. Sun, T. Zhang, D. Zou, ABPT: Amended backpropagation through time with partially differentiable rewards, arXiv preprint arXiv:2501.14513 (2025).

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347 (2017).

[29] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, S. Levine, Soft actor-critic algorithms and applications, arXiv preprint arXiv:1812.05905 (2018).

[30] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, P. Abbeel, Trust region policy optimization, arXiv preprint arXiv:1502.05477 (2015).

[31] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, arXiv preprint arXiv:1506.02438 (2015).

[32] T. Haarnoja, H. Tang, P. Abbeel, S. Levine, Reinforcement learning with deep energy-based policies, in: Proc. International Conference on Machine Learning (ICML), 2017, arXiv:1702.08165.

[33] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Proc. International Conference on Learning Representations (ICLR), 2015, arXiv:1412.6980.

[34] S. Peitz, J. Stenner, V. Chidananda, O. Wallscheid, S. L. Brunton, K. Taira, Distributed control of partial differential equations using convolutional reinforcement learning, Physica D: Nonlinear PhenomenaArXiv:2301.10737 (2024).

[35] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, JAX: Composable transformations of Python+NumPy programs, https://github.com/jax-ml/jax (2018).

[36] S. Fujimoto, H. van Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: Proc. International Conference on Machine Learning (ICML), 2018, pp. 1582–1591, arXiv:1802.09477.

[37] D. J. Rezende, S. Mohamed, D. Wierstra, Stochastic backpropagation and approximate inference in deep generative models, in: Proc. International Conference on Machine Learning (ICML), 2014, pp. 1278–1286, arXiv:1401.4082.

[38] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, in: Proc. International Conference on Learning Representations (ICLR), 2016, arXiv:1509.02971.

[39] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, in: Proc. International Conference on Learning Representations (ICLR), 2016, arXiv:1511.05952.

[40] P. J. Werbos, Backpropagation through time: What it does and how to do it, Proceedings of the IEEE 78 (10) (1990) 1550–1560. doi:10.1109/5.58337.

[41] P. Parmas, M. Sugiyama, Total stochastic gradient algorithms and applications in reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2018, arXiv:1902.01722.

[42] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, H. Larochelle, Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program), Journal of Machine Learning Research 22 (164) (2021) 1–20.

Cem Özkurt is an Assistant Professor in the Department of Data Science and Analytics, Faculty of Computer and Information Sciences, Sakarya University, and Director of the Sakarya University National Technology Workshop Artificial Intelligence Center. He received his Ph.D. in Industrial Engineering from Sakarya University in 2017. His research focuses on artificial intelligence and machine learning, Industry 4.0 and digital transformation, and data analytics in manufacturing and defense industries. In this study he conceived the research, supervised its methodology and administered the project.

Muhammed Emin Onay received his B.Sc. degree in Mechatronics Engineering from Sakarya University of Applied Sciences and is currently pursuing an M.Sc. degree in the Department of Computer Engineering, Faculty of Computer and Information Sciences, Sakarya University. He is a researcher and engineer at the Sakarya University National Technology Workshop Artificial Intelligence Center. His research interests include deep learning, reinforcement learning, and robotics. In this study he contributed to data curation, visualisation and the experimental investigation.

Eyüp Altuğ Tunç received his B.Sc. degree in Mechatronics Engineering from Sakarya University of Applied Sciences and is currently pursuing an M.Sc. degree in the Department of Computer Engineering, Faculty of Computer and Information Sciences, Sakarya University. He is a researcher at the Sakarya University National Technology Workshop Artificial Intelligence Center. His research interests include artificial intelligence, control systems, and autonomous robotics. In this study he contributed to the software implementation, validation and the experimental investigation.

Ahmet Kutay Küçükler is a student in the Department of Computer Engineering, Sakarya University of Applied Sciences. He has worked as a research student at artificial intelligence research centers in Sakarya, including the Sakarya University National Technology Workshop Artificial Intelligence Center. His research interests include artificial intelligence and data analytics. In this study he contributed to the formal analysis, the provision of resources and the validation of the results.

Haktan Yusuf Güloğlu is an undergraduate student in the Department of Computer Engineering, Sakarya University of Applied Sciences, and a student researcher at the Sakarya University National Technology Workshop Artificial Intelligence Center. His research centers on reinforcement learning and large language models, with a particular interest in the stability of learning algorithms under complex, high-dimensional, and chaotic dynamics. In this study he contributed to the software implementation and methodology and wrote the original draft of the manuscript.